INDEX

ANITS

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1.	1/7	Introduction & basic HTML	CO_1		
2.	8/7	Design HTML using audio	CO_1		
3	15/7	Design HTML using video	CO_1		
4.	22/7	program using JS	CO_1		
5.	29/7	Develop dynamic webpage	CO_1		
6.	5/8	Implement modules in nodeJS	CO_2		
7.	12/8	mini application using express & nodeJS	CO_2		
8.	19/8	Code Tantra (static webpage)	CO_1		
9.	2/9	Code Tantra (portfolio)	CO_1		
10.	9/9	Code Tantra (simple calculator)	CO_1		
11.	16/09	Create a function & class	CO_2		
12.	23/09	Implement http methods	CO_2		
13.	29/09	Develop CRUD operation	CO_3		

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-2027	Semester:	I
Name of the Student:	P Karthikeya	Roll Number:	A24126552167
Name of the Lab:	FSWBD Lab	Timings:	9:40-12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva- Voce (2Marks)			
1/7/26	1					11		
8/7/26	2					11		
15/7/26	3					12		
22/7/26	4					12		
29/7/26	5					12		
5/8/26	6					11		
12/8/26	7	4	4	5	0	13		
19/8/26	8							
02/9/26	9							
9/9/26	10							
16/9/26	11							
23/9/26	12							
30/9/26	13							

Name of the Faculty:

Faculty Signature:

01/07/26

Introduction to HTML and CSS

⇒ HTML (Hyper Text Markup Language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a web page using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by the web browsers to display the content in a structured and meaningful manners. HTML provides the structure and semantic organization of a web page.

⇒ CSS (cascading style sheet) is a system stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, back grounds, spacing, borders, alignment, dimensions and layers. CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices. CSS can be applied using inline, internal or external stylesheets

=) HTML and CSS work together to develop webpages where HTML provides the structure and content while CSS provides the styling layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized, attractive and user-friendly web interfaces.

Difference between MERN stack and full stack

=) MERN stack is a java script based technology stack used for developing modern web applications. MERN stands for Mongo DB, Express.js, React.js, Node.js. Mongo DB is used for database management, express.js handles the backend framework and API's. React.js is used to develop frontend users in interface and node.js provides the server side runtime environment.

⇒ Full Stack Development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic, managing databases.

8/7/26

HTML Components :-

1) HTML Tags - define the structure and elements of a webpage
2) Headings - used to display titles and heading using <h1> to <h6>
3) paragraphs - used to display text using the <P> tag.
4) Links - used to navigate btw webpages using <a> tag
5) Images - used to display images using <img> tag
6) Lists - used to create ordered and unordered lists using <ol> ,
7) Tables - used to organize data in rows and columns
8) Forms - used to collect user input using elements such as text boxes, buttons and checkboxes
9) Div and span - used to group and organize webpage content
10) semantic Elements - Elements such as <header>, <nav>, <select>, <articles>, <footer> provide meaningful webpage structure

CSS Components :-

1) Selectors - Select HTML elements to apply Styles

2) Properties - Define the visual characterstics of elements

3) Values - Specify the setting of a CSS property

4) Colors - Define text, background, borders, colors.

5) Fonts - control font family, size, Style and weight.

6) Backgrounds - Size background colors and images

7) Borders - Define the borders style, width and radius

8) Margin - controls the space outside an element

9) padding - controls the space inside element

10) Layout - controls the arrangement of elements using flexbox, Grid and positioning

HTML vs HTML5

HTML	HTML 5
→ older version of HTML	→ Latest major versi of HTML
→ Limited multimedia Support	→ support audio and video
→ uses external playing for multimedia	→ No external player needed
→ Has fewer semantic tags	→ provides semantic tags
→ No <canvas> elements	→ Supports <canvas> elemen
→ Limited web storage	→ Supports Local Storage
→ Complex Doctype declaration	→ Simple <!DOCTYPE HTML>

CSS vs CSS 3

CSS	CSS 3
→ Earlier version of CSS	→ Advanced version of CSS
→ Basic styling features	→ provides advanced Styling features
→ Limited animations	→ Supports animations and transitions
→ No flexible layout Systems	→ Supports flexbox and Grid
→ Limited visual effects	→ supports shadows, gradients, effects
→ No media queries	→ Supports media queries
→ uses a simple specification	→ Divided into different modules

15/7/26

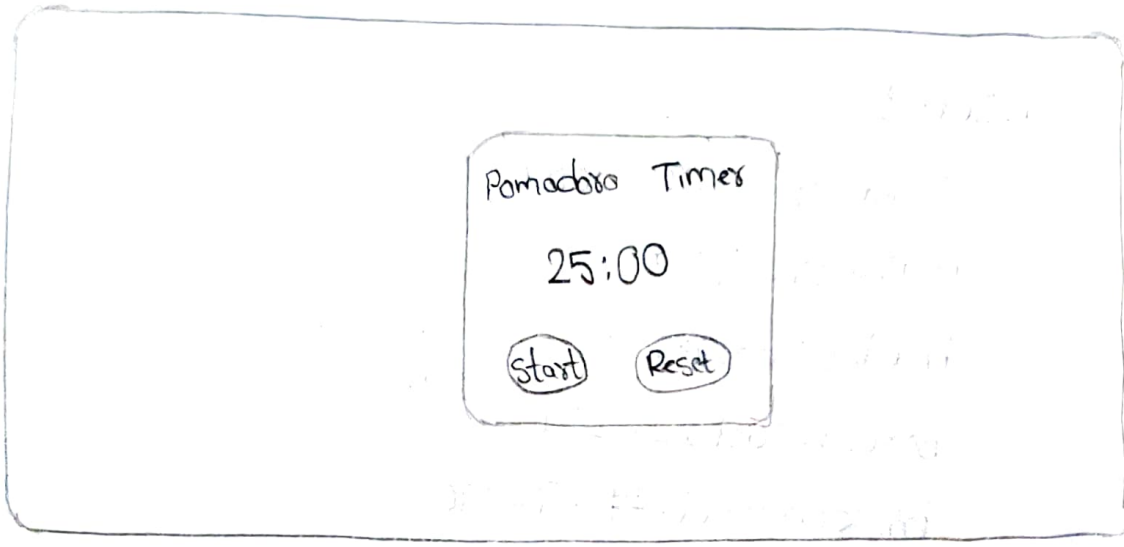
e) Design a static web page using HTML and CSS

Pomodoro Timer, using HTML CSS

```
<!DOCTYPE HTML>
<html>
  <head>
    <title> Promodorotimer </title>
    <style>
      body {
        font-family: trial;
        background: # f5f5f5;
        text_align: center;
        padding: 50px;
      }
      timer {
        width: 300px;
        margin: auto;
        Padding: 30px;
        background: white
        border_radius: 15px;
        box_shadow: 0 0 10px gray;
      }
    hi {
      color: #c74c3c;
    }
```

```
time {
    font-size: 50 px;
    margin: 20px 0;
}
button {
    Padding: 10px 20px;
    margin: 5px;
    border: none;
    border-radius: 5 px;
    background: # P74C3C;
     color: white;
}
</style>
</head>
<body>
  <div class="Timer">
     <h1> Promodore timer </h1>
     <div class = "time"> 25:00 </div>
     <button> Start </button>
     <button> Reset </button>
  </div>
</body>
</html>
```

o/p :-



22/7/26

(i) Design an HTML code using audio tags

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My Favorite Song </h2>
    <audio controls>
        <source src = "song.mp3" type = "audio/mpeg">
        Your browser does not support the audio tag
    </audio>
</body>
</html>
```

o/p:-

Displays an audio player with play, pause and volume controls.

Steps:-

- Create a new HTML file
- Add the basic HTML structure
- Add heading using <h2> tag
- Insert the <audio> tag with controls attribute
- Specify the audio file using <source> tag
- Save and open the HTML file in a web browser
- play the audio using displayed controls.

(4) Design a HTML code using video tags

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> My Video </h2>
    <video width = "400" controls>
        <source src = "video.mp4" type = "video/mp4">
    </video>
</body>
</html>
```

o/p :-

Displays a video player with play, pause, volume and Fullscreen controls.

Steps:

- Create a new HTML file
- Add a basic HTML structure
- Add a heading using the <h2> tag
- Insert the <video> tag with controls attribute
- Specify the video file using <source> tag
- Save the file and open it in web browser
- play the video using displayed controls.

(iii) Write a program using JavaScript (arrays, functions)

code:

```
let numbers= [12,45,8,23,56,19,34,67,90,11];
function displayArray(arr){
    Console.log("Array Elements: ");
    for (let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}
function findSum(arr){
    let sum=0;
    for(let i=0; i<arr.length; i++){
        Sum+=arr[i];
    }
    return sum;
}.
```

o/p:-

Array Elements:

12,45,8,23,56,19,34,67,90,11

Sum: 365

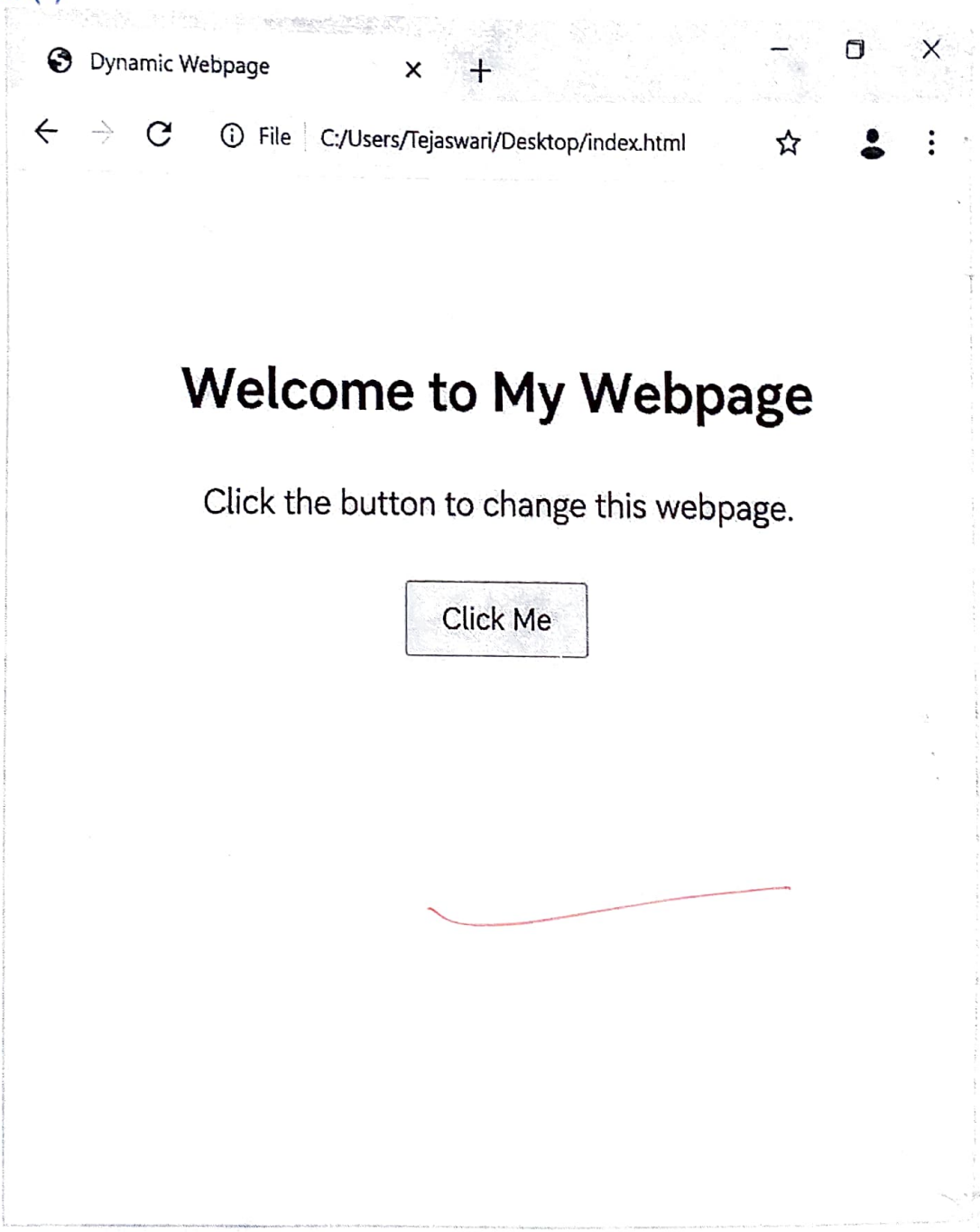
Experiment - 04

4) Develop dynamic webpage using Javascript

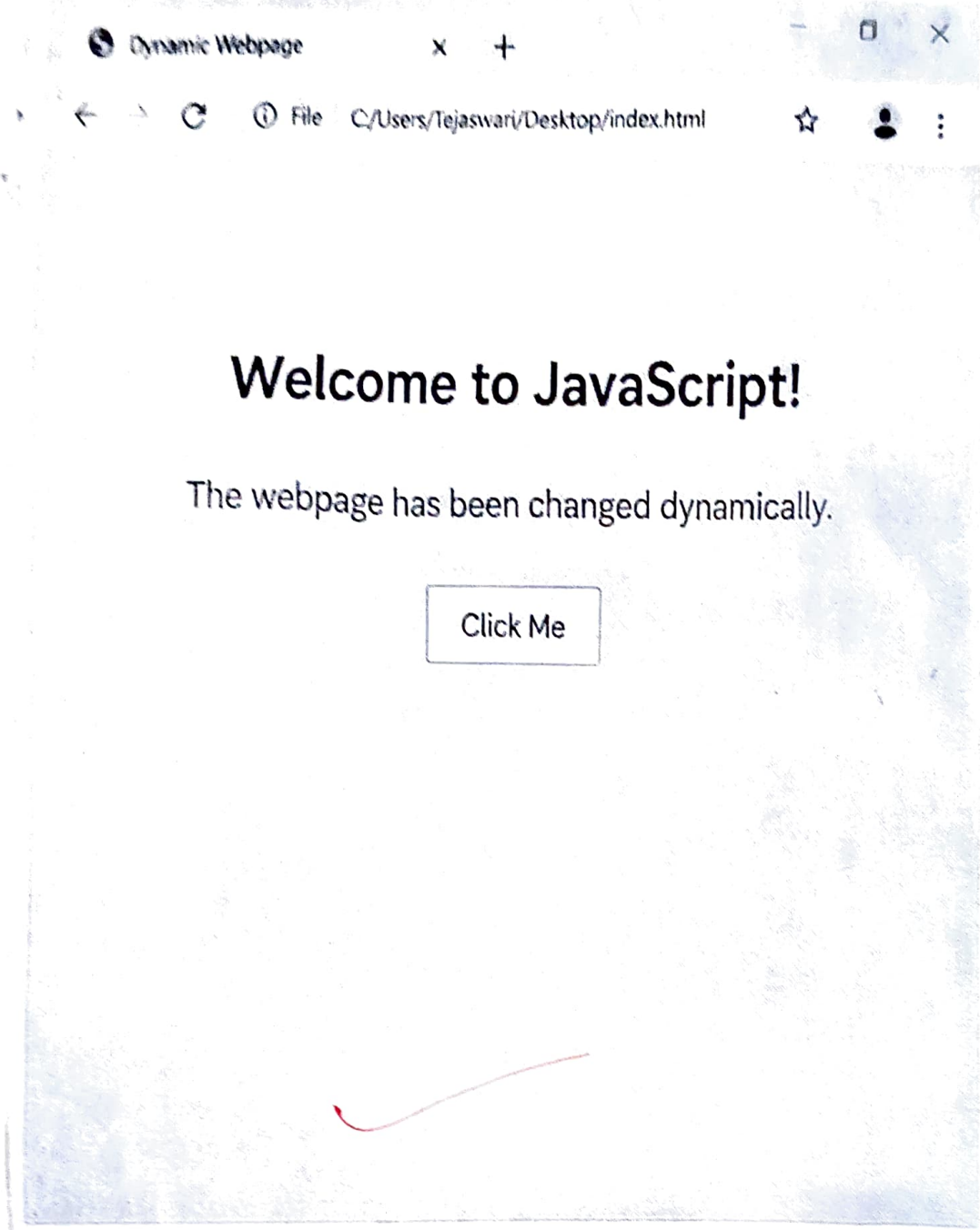
Program:-

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic webpage</title>
  <style>
    body{
      text-align: center;
      font-family: Arial;
      margin-top: 100px;
    }
    button {
      Padding: 10px 20px;
      font-size: 16px;
      cursor: pointer;
    }
  </style>
</hed>
<body>
  <h1 id = "heading">Welcome to My webpage</h1>
  <p id = "message"> click the button to
      change this webpage </p>
  <button onclick = "change page()"> clickMe
                                    </button>
  <script>
    function changepage(){
      document.getElementById ("heading");
```

```
innerHTML = "Welcome to JavaScript";
document.getElementById("heading")
innerHTML = "The Webpage has been
changed dynamically"
document.body.style.backgroundcolor = "light blue"
{
</script>
</body>
</h
```



Before clicking the button



After clicking the button

Experiment - 05

Implement Modules in Node.JS

- In Node.js, you can implement modules using from distinct systems: CommonJS, which is traditional default, and Es modules (ESM), the modern JavaScript standard

1. The CommonJS Method (default):

CommonJS uses module exports to share code and require() to import it. This system loads modules synchronously.

a. Create the module (math.js):

```
const add = (a,b) => (a+b);
const subtract = (a,b) => a-b;
module.exports = {
        add,
        Subtract
};
```

b. Use the module (app.js):

```
const math = require ('./math');
console.log (math.add (10,5));
console.log (math.subtract (10,5));
```

O/P:-

```
15
5
```

2. The ES modules Method (modern standard) :-

ES modules use export to share code and import to use it. To make Node.js recognize ES modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a) Create the module (calculator.js)

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorName = "supercase";
export default calculatorName;
```

b) Use the module (main.js)

```
import calculatorName, {multiply, divide} from
                    './calculator.js';

console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

o/p: supercalc
8
2

3) Using Built-in core modules:

Node.js also comes equipped with dozen of built-in core modules (like fs for file system or path for file working) that doesn't need installation.

we will load them by module names:

```
// common JS style
const fs = require('fs');
// ES modules style
import fs from 'fs';
```

Here are some of the most commonly used ones:

1. fs = file system operations
2. http = HTTP server and client
3. path = File path utilities
4. os = operating system utilities
5. events - Event handling
6. util - utility functions
7. stream - stream handling
8. crypto - cryptographic functions
9. url - URL parsing
10. query string - URL query string handling.

Eg: Simple HTTP server

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.end('Hello world!');
}).listen(8080);
```

O/P: Hello world!

Experiment -6

6) Develop mini application using express and NodeJS

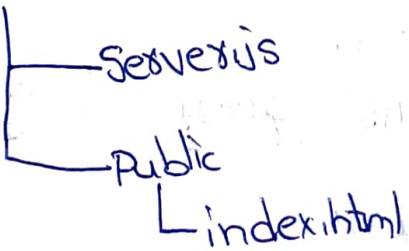
- Simple mini application using nodeJS, ExpressJS

Student Management System:-

1. Add a student
2. View all students
3. delete a student
4. Uses Express.js routes
5. Simple HTML interface

Project structure:-

Student.app/



Server.js:-

```
Const express = require("express");
Const app = express()
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("Public"));
```

```
pp.get ("/students", (req,res) => {
    res.json(students);
});
//Add
app.post("/students", (req,res) => {
    const student = {
        id: students.length+1,
        name: req.body.name,
        course: req.body.course
    };
    Student.push(student);
    res.json ({message: "Student added
                        successfully",
                student: student });
});
// Delete
app.delete ("/students/:id", (req, res) => {
    const id = parseInt (req.params.id);
    Students= students.filter (student =>
            student.id!==id);

    res.json({
        message: "Student deleted
                        successfully"});
});
```

```
//start Server
app.list (PORT, () => {
    console.log ('Server running at http://localh...
        ${PORT}');
});
```

Public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title> Student Management </title>
</head>
<body>
    <h1> Student Management System </h1>
    <input type="text" id="name"
    placeholder="Enter student name">
    <input type="text" id="course"
    placeholder="Enter course">
    <button onclick="addStudent()">Addstuden...
        </button>
    <h2> Student List </h2>
```

```
</ul id="studentlist"></ul>
</script>
  function addStudent(){
    let name= document.getElementById
              ("name").value;
    let course=document.getElementById
              ("course").value;
    fetch("/students",{
        method:"POST",
        headers:{
          "Content-Type":"application/json"},
        body: JSON.stringify({
              name:name,
              course:course
        })
    })
    .then(response => response.json())
    .then(data => {
          alert(data.message);
document.getElementById("name").value="";
document.getElementById("course").value="";
    displayStudents();
  });
}
```

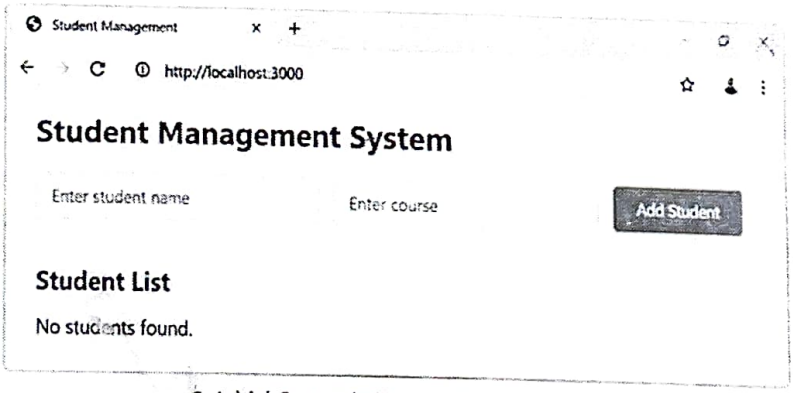
```
function displayStudents(){
  fetch("/students").then(response => response.json())
  .then(students => {
    let list = document.getElementById("student
                                        list");
    list.innerHTML= "";
  students.forEach(student => { let li=
      document.CreateElement("li");
          li.innerHTML= student.id + "-" +
                        student.name + "-" +
                        student.course + "</button
  onclick = 'deletestudent("+ student.id +")'>Dele
  </button>";
  list.appendChild(li);
              });
          });
    }
    function deletestudent(id){
        fetch("/students/"+id, {method "DELE
  .then(response => response.json())
  .then(data => {
          alert(data.message);
          displayStudents();
  });
```

```
}
displayStudents();
</script>
</body>
</html>
```

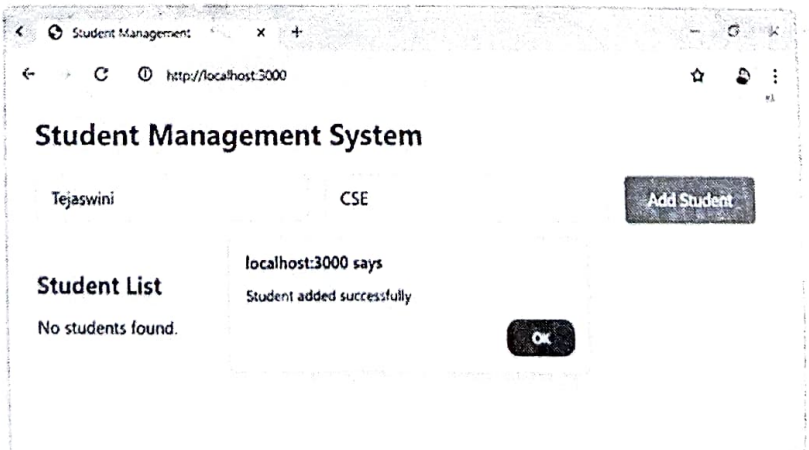
o/p:-



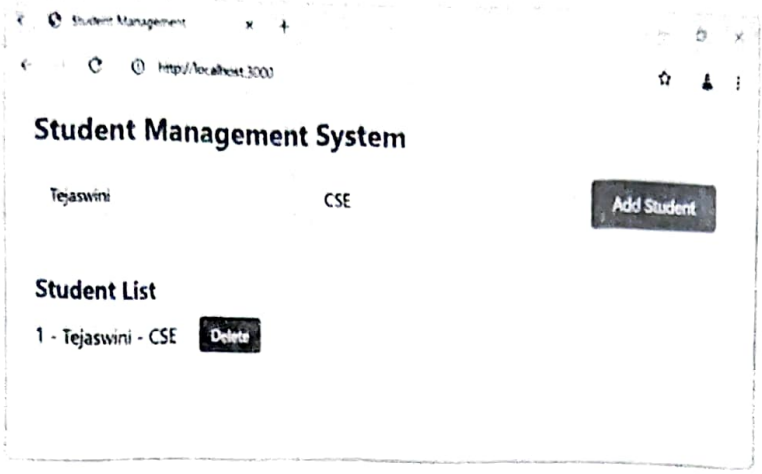
1. Terminal Output (after running `node server.js`)



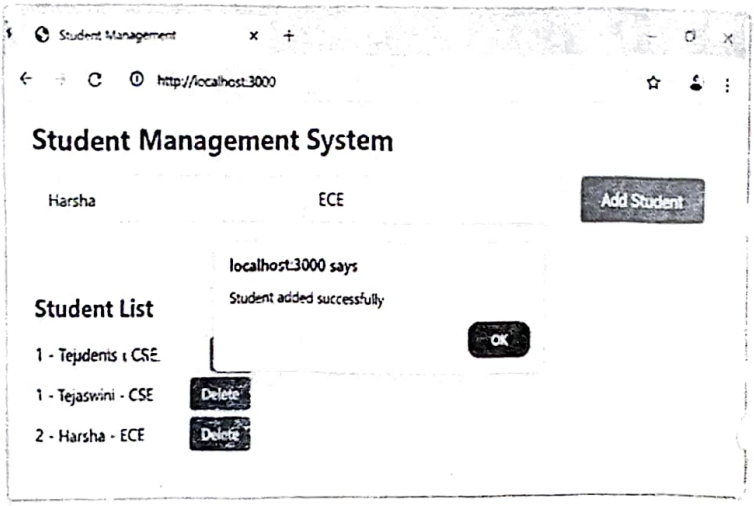
2. Initial Output in Browser (no students)



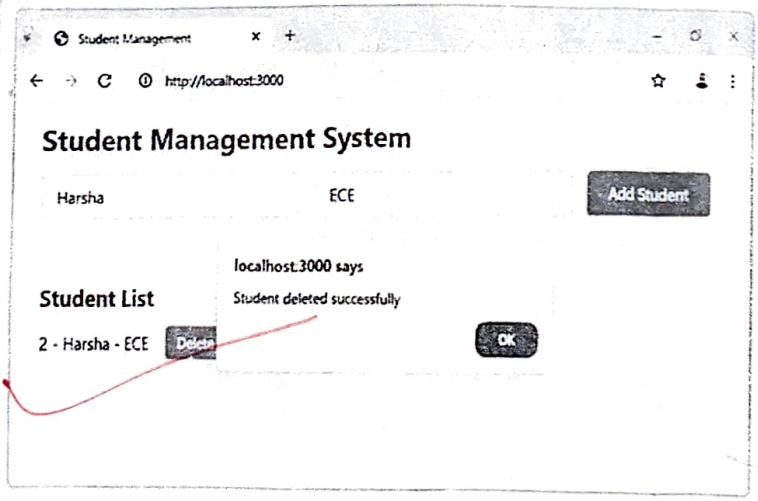
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment-07

7. Create a functional component and cl
component using react.

Aim:- To develop a react application us
Functional components and class compon
to display student details, manage state
handle button clicks and dynamically
update data on a web page.

- Functional Component:-

A functional component is a JavaS
function that returns JSX to display t
user interface. It uses React Hooks such
usestate to manage state and update th
webpage dynamically.

Program:-

```
import React, {useState} from
'react';
function studentFunctional(){
    const [name, setName] = useState ("Tejaswa
    const [course, setCourse] = useState ("CSM");
    const [count, setCount] = usestate(0);
    const changeName = () => {
        setName("Teju");
```

```
Const changeCourse = ()=>{
    setCourse("CSD");
};
Const incrementCount = ()=>{
    setCount(count+1);
};
return(
  <div style={{
      border= "8px solid blue",
      Padding: "20px",
      margin: "20px",
      borderRadio: "10px",
      textAlign: "center"
    }}>
  <h1> Student Details </h1>
  <h2> Functional Component </h2>
  <p> <b> Name: </b> {name} </p>
  <p> <b> Course: </b> {course} </p>
  <p> <b> Button Click: </b> {count} </p>
  <button onClick={changeName}>
      changeName </button>
  <button onClick={changeCourse}> change
      course </button>
```

```
        <button onclick={incrementCount}>Increment</button>
    </div>
);
```

dp:

When the user clicks the buttons, the name, course and count change dynamically without reloading the webpage.

Class component:

A class component is an JavaScript class that extends React Component. It the render() method to return JSX and manages state using this.state and this.setState().

Program:

```
import React, {Component} from 'react';
  class studentclass extends Component{
     constructor(props){
        super(props);
          this.state = {
              name: "Tejaswori",
              course: "computer science",
              count: 0
          };
      }
     changeName = () => {
        this.setState({
            name: "Teju"
          });
        };
     changeCourse = () => {
        this.setState({
            course: "CSM"
        });
    };
```

```
incrementCount = ()=>{
    this.setState((PrevState)=>({
        Count: prevState.count+1
    }));
};
render(){
  return(
    <div style={{
        border: "2px solid green",
        Padding: "80px",
        margin: "20px",
        borderRadius: "10px",
        textAlign: "center"
    }}>
    <h1>Student Details</h1>
    <h2>class component</h2>
<p><b>Name:</b>{this.state.name}</p>
<p><b>Course:</b>{this.state.course}</p>
<p><b>Button clicks</b>{this.state.count}</p>
<button onclick={this.changeName}>
        changeName</button>
<button onclick={this.changeCourse}>
        changeCourse</button>
```

on onclick={this.incrementCount}>
Increment </button>
div>

Experiment-08

8: Implement http methods
Server

t App

localhost:3000

React Components

Student Details

Functional Component

Name: Tejaswari
Course: CSE
Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Tejaswari
Course: CSE
Button Clicks: 0

Change Name | Change Course | Increment

the following Comm

Experiment-08

8. Implement http methods using JSON Server

Aim:- To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD operations on student records.

Theory:- JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate between a client and a Server.

- GET - Retrieves data from the server
- POST - Adds new data to the Server
- PUT - Replaces an existing record
- PATCH - updates selected fields of record
- DELETE - Removes a record from the Server.

a. Create a JSON Server

Step1: Initialize the project

Open the vscode terminal and execute the following commands:

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install json server
```

Step 2: Create db.json

Create a file named db.json and add the following student data

```
{
  "students": [
    {
      "id": "1",
      "name": "Tejaswari",
      "Course": "CSM",
      "marks": "90"
    },
    {
      "id": "2",
      "name": "chaitrika",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "3"
      "name": "sravani",
      "Course": "IT",
      "marks": 88
    }
```

Step 3: Start the JSON Server

For the current JSON Server version, execute:

```
npx json-server db.json
```

The server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students

2. Implementation of HTTP methods

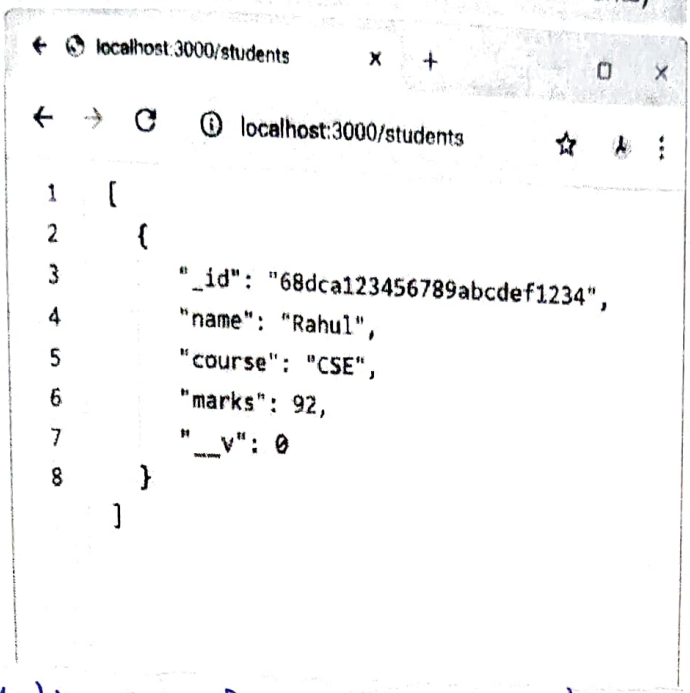
a. GET method: The GET method is used to retrieve student records from JSON server

Code:-

```
fetch('http://localhost:3000/students')
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

O/P :-

1. GET All Students (http://localhost:3000/students)



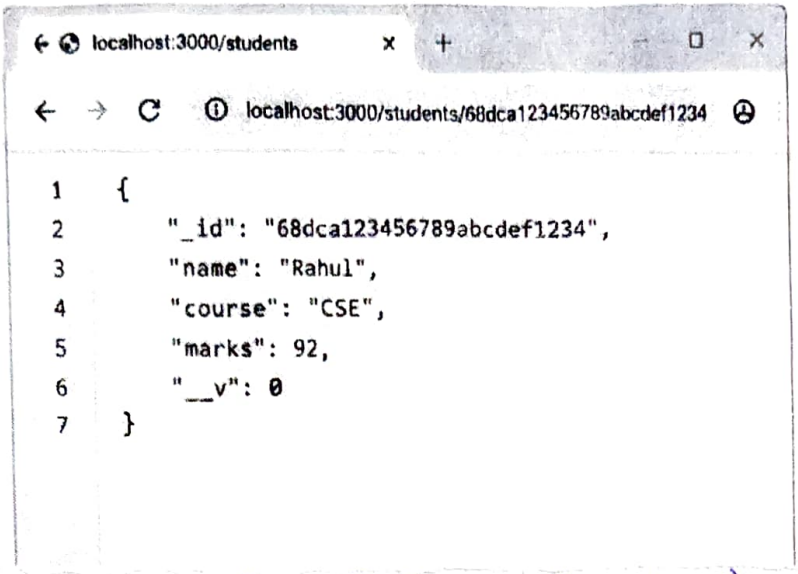
B. POST Method: The POST method is use to insert a new student record into the server

code :-

```
fetch ("http://localhost:3000/students', {
        method: 'POST',
        headers: {
            'Content-Type' : 'Application/json'
        },
        body: JSON.stringify ({
            name: 'Rahul',
            Course: 'CSE',
            marks: 92
        })
    })
    .then (response => response.json())
    .then(data => {
            console.log(data);
    })
```

```
.catch (error => {
    console.log(error);
});
```

o/p:-



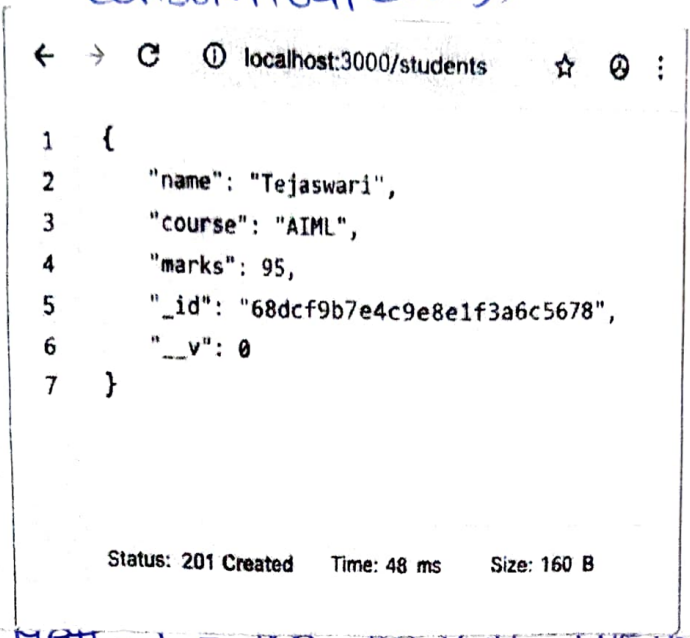
C. PUT Method: The PUT method is used to replace an existing student record with updated data.

Code:-

```
fetch('http://localhost:3000/students/1', {
    method: 'PUT',
    headers: {
        'content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Tejaswari',
        course: 'AINL',
        marks: 95
    })
})
.then(response => response.json())
```

```
.then (data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

o/p:-



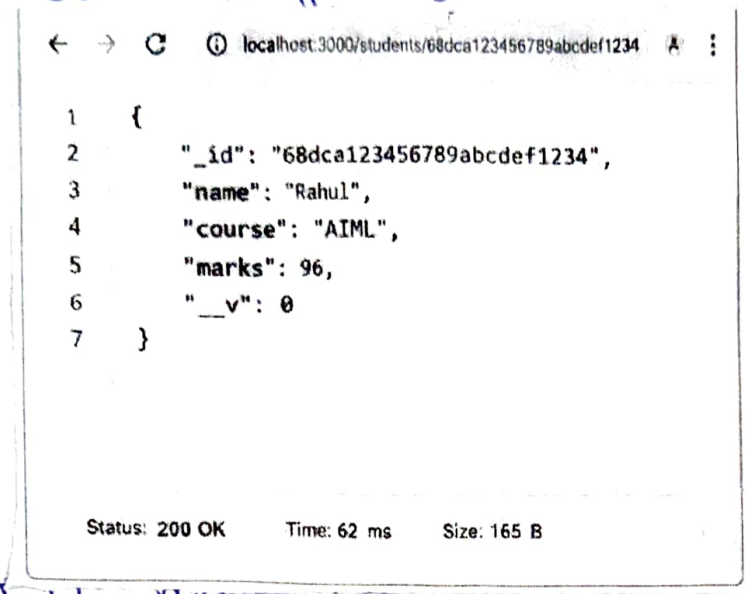
D. PATCH Method - The PATCH method is used to update only selected fields of an existing student record.

Code:-

```
fetch ('http://localhost:3000/students/2', {
    method : 'PATCH',
    headers: {
        'Content-Type' : 'application/json'
    },
    body: JSON.stringify({
        marks:96
    })
})
.then (respons => response.json())
.then (data => {
```

```
    console.log(data);
});
.catch(error => {
    console.log(error);
});
```

o/p:-



E. DELETE method: The DELETE method is used to remove an existing student record from the server.

Code:-

```
fetch('http://localhost:3000/students/3', {
    method: 'DELETE'
})
.then(response => {
    if(response.ok) {
        console.log("Student deleted successfully");
    }
})
.catch(error => {
    console.log(error)
});
```

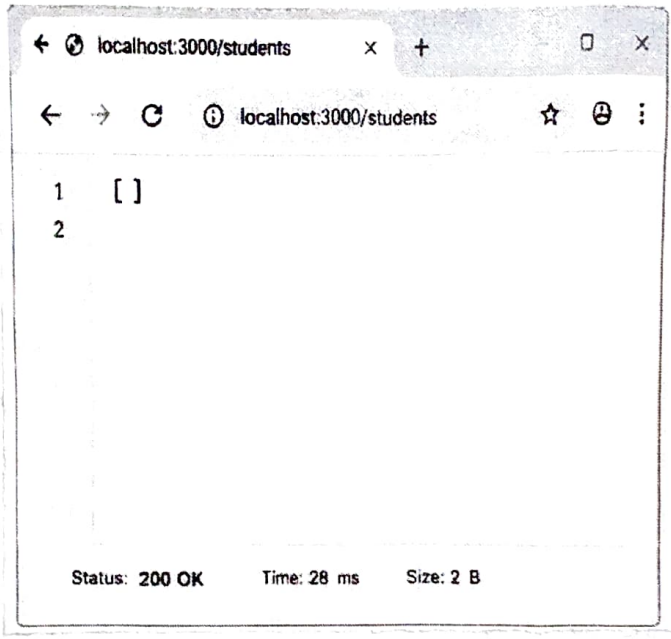
o/p :-

```
{
    "message": "Student deleted successfully"
}
```

Status: 200 OK Time: 45 ms Size: 46 B

- After i and DELETE operations in sequence, the remaining student will be:

o/p :-



Thus, HTTP methods were successfully implemented using JSON server. The CRUD operations were performed on student record the data was retrieved, inserted, updated and deleted using HTTP requests

Experiment-9

9. Develop CRUD operations using MongoDB

Aim:- To develop a web application using Node.js, Express.js and MongoDB to perform CRUD operations on student records.

Theory: MongoDB is a NoSQL database, that stores data in flexible, JSON-like documents, CRUD operations are used to manage data in a ~~stru~~ database.

Mongoose is a object Data Modeling library for MongoDB and Node.js. It helps define schemas, interact with Collections and Perform database operations.

1. Installation and project setup:

```
mkdir MongoDB_CRUD
cd MongoDB_CRUD
npm init -y
npm install express mongoose.
```

2. Program: Server.js

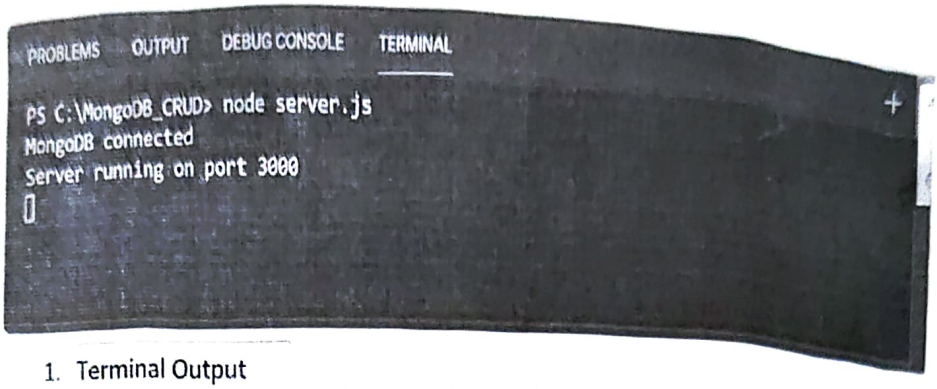
```
Const express = require('express');
const mongoose = require('mongoose');
Const app = express();
Const PORT = 3000;
```

```
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:27017/
                                    college')
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
//create student
const StudentSchema=new
mongoose.Schema({
    name: {
        type: String,
        required :true
    }
    Course: {
        type: String,
        required :true,
    },
    marks: {
        type: Number
        required :true
    }
});
// Create student model
Const Student:
    mongoose.model('student', StudentSchema)
```

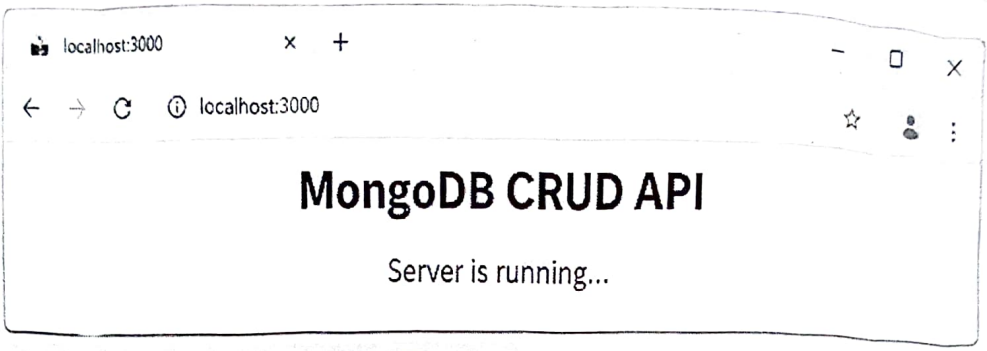
```
// CREATE: Add
app.post('/students', async (req, res) => {
  try {
    const student = new Student(req.body);
    Const SavedStudent = await StudentSave();
  res.status(201).json(savestudent);
  } catch (error) {
      res.status(400).json({ message
                  error: message});
      }
  });
  // READ: Get all students
app.get('/students', async (req, res) => {
  try {
    Const students = await
    Student.find();
    res.status(500).json({ message: error
                  .message });
    }
});
// READ: Get student by ID
app.get('/students/id', async (req, res) => {
  try {
```

```
Const student = await
 Student.findBId (req.Params id);
 if (!student){
     return res.status(404).json{{
         message: "student not found"
     });
  }
  res.status(200).json(student
 } catch (error){
    res.status(400).json {{ message: error.
                          message }):
  }
});
// UPDATE
app.put ('/students/id: async (req, res) => {
  try{
    const student = await
    student  findByIdAnd Update (
        req.params.id,
        req.body,
        {new :true, runValidators: true}
      );
    if (! student){
       return res: status(404).json( {
```

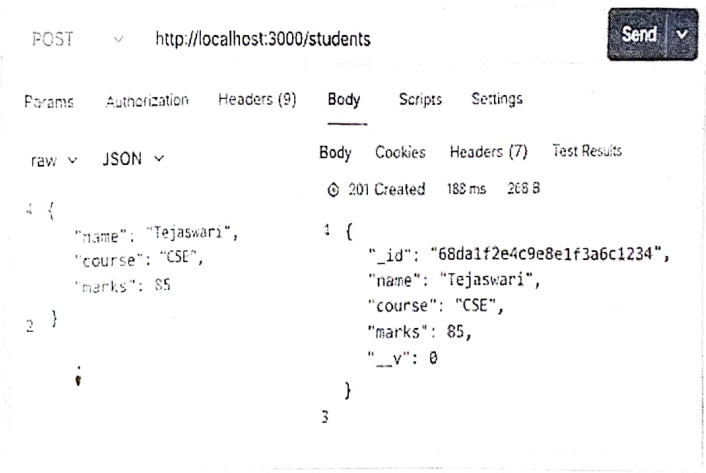
```
            message: "Student not found"
        });
    }
    res.status(200).json(student);
    } catch (error) {
        res.status(400).json({message:
                                error.message});
    }
});
//DELETE
app.delete('/students/:id', async(req,res)=>{
    try{
        Const student await student,
                findByIdAndDELETE(
            req.params.id
        );
    if(!student){
        return res.status(404).json({
            message: "Student not found"
        });
    }
    res.status(200).json({
        message: "Student deleted successfully"
    });
    } catch (error) {
        res.status(400).json({message: error.message});
});
```



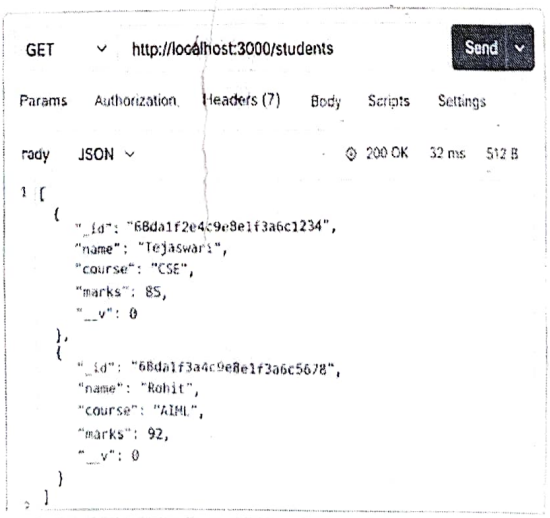
1. Terminal Output



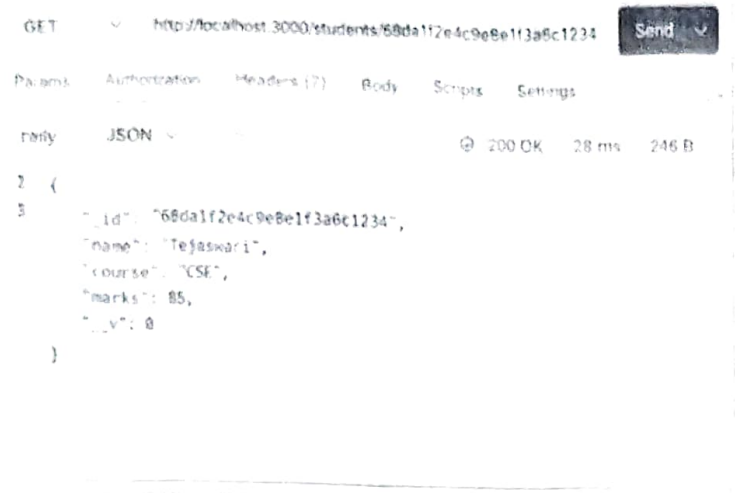
2. Chrome Browser (Home Page)



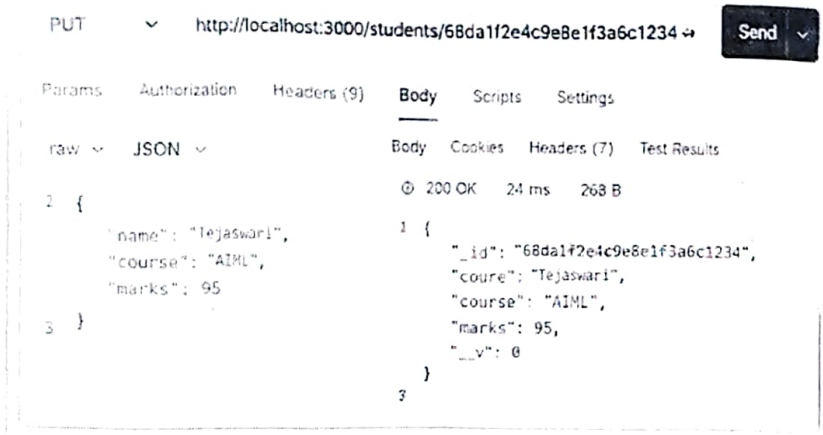
3. Create (POST) – Add Student



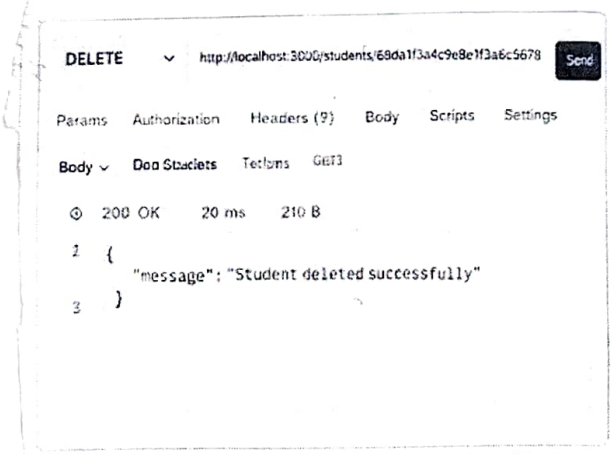
4. Read (GET) – Get All Students



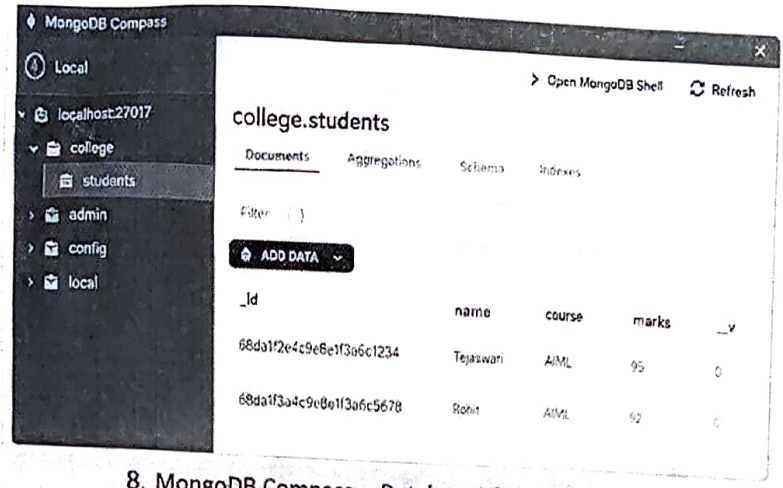
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View